

Documentação Técnica — FarmacoMatch

Verificação de compatibilidade medicamentosa Versão do documento: 1.5

1. Visão geral

O **FarmacoMatch** é uma aplicação web de página única (SPA) construída com **Streamlit** que permite ao usuário selecionar dois medicamentos e verificar se existe alguma contraindicação registrada para o uso concomitante desses medicamentos.

A verificação não é feita diretamente entre medicamentos, mas sim entre as **classes farmacológicas** às quais cada medicamento pertence. A aplicação consulta um banco de dados SQLite local (`med.db`) que mapeia:

- **Medicamento → Classe farmacológica** (tabela `medicamentos`)
- **Par de classes → Risco/contraindicação** (tabela `interacao_classes`)

Quando não há registro de interação entre as classes dos dois medicamentos selecionados, a aplicação exibe um card de sucesso ("Pode misturar"). Caso exista registro, exibe um card de erro ("Não pode misturar") com a descrição do risco.

Público-alvo

- Profissionais de saúde e estudantes que precisam de uma verificação rápida de interações medicamentosas.
- Projeto acadêmico (Unifil) ilustrando integração entre Python, Streamlit e SQLite.

2. Tecnologias e dependências

Tecnologia	Papel no projeto
Python 3	Linguagem base
Streamlit	Framework web para a interface
SQLite3	Biblioteca padrão do Python para acesso ao banco local

`requirements.txt`

```
streamlit
```

O SQLite3 é parte da biblioteca padrão do Python e não precisa ser declarado em `requirements.txt`.

3. Estrutura de arquivos

```
FarmacoMatch/  
├─ app.py          # Aplicação Streamlit (UI + lógica + acesso a dados)
```

```
└─ med.db          # Banco de dados SQLite (medicamentos e interações)
└─ requirements.txt # Dependências Python
└─ README.md       # (vazio-placeholder)
└─ venv/           # Ambiente virtual (não versionado)
└─ .idea/          # Configuração do IDE JetBrains
```

A aplicação inteira reside em um único arquivo `app.py`, seguindo o padrão monolítico típico de protótipos em Streamlit.

4. Modelo de dados

O banco `med.db` contém **2 tabelas** e **4 índices**.

4.1 Tabela `medicamentos`

Armazena os medicamentos cadastrados e suas respectivas classes farmacológicas.

Coluna	Tipo	Descrição
id	INTEGER	Chave primária
nome	TEXT	Nome do medicamento
classificacao	TEXT	Classe farmacológica (ex.: ANALGESICOS, ANTIBIOTICOS)

Volume: 936 registros.

Índices: - `name_idx` → `medicamentos(nome)` - `class_name_idx` → `medicamentos(classificacao)`

4.2 Tabela `interacao_classes`

Armazena as interações (contraindicações) entre pares de classes farmacológicas.

Coluna	Tipo	Descrição
id	INTEGER	Chave primária autoincrement
class_1	TEXT	Primeira classe da interação
class_2	TEXT	Segunda classe da interação
risco	TEXT	Descrição do risco/contraindicação

Restrição: `UNIQUE(class_1, class_2)` — cada par de classes aparece uma única vez.

Volume: 1930 registros.

Índices: - `int_class1_name` → `interacao_classes(class_1)` - `int_class2_name` → `interacao_classes(class_2)`

4.3 Diagrama do modelo (texto)





A relação entre as tabelas é **lógica** (via `classificacao` ↔ `class_1/class_2`); não há chave estrangeira física declarada.

4.4 Classes farmacológicas de exemplo

O banco contém dezenas de classes distintas, entre elas:

- ANALGESICOS
- ANTIBIOTICOS
- ANTI-INFLAMATORIOS
- ANTI-HIPERTENSIVOS
- ANTIALERGICOS
- ANTICONVULSIVANTES
- ANTIDEPRESSIVOS
- ANTIDIABETICOS
- ANSIOLITICOS
- ANTI-FUNGICOS

5. Funções da aplicação

O `app.py` expõe três funções de acesso a dados, todas no início do arquivo:

5.1 `busca_lista_medicamentos()`

Localização: `app.py:12`

Responsabilidade: retornar a lista de todos os nomes de medicamentos cadastrados, em ordem alfabética, para alimentar os `selectbox` da UI.

Retorno: `list[str]` — nomes dos medicamentos.

```

def busca_lista_medicamentos():
    conn = sqlite3.connect('med.db')
    cursor = conn.cursor()
    cursor.execute("SELECT nome FROM medicamentos ORDER BY nome")
    lista = cursor.fetchall()
    conn.close()
    return [row[0] for row in lista]

```

5.2 `busca_classificacao_remedio(nome_remedio: str) -> str | None`

Localização: `app.py:20`

Responsabilidade: dado o nome de um medicamento, retornar sua classificação farmacológica.

Retorno: `str` com a classificação, ou `None` se o medicamento não existir.

```
def busca_classificacao_remedio(nome_remedio: str) -> str | None:
    ...
    return classificacao_remedio
```

5.3 busca_contra_indicacao(classificacao_1, classificacao_2) -> str | None

Localização: `app.py:40`

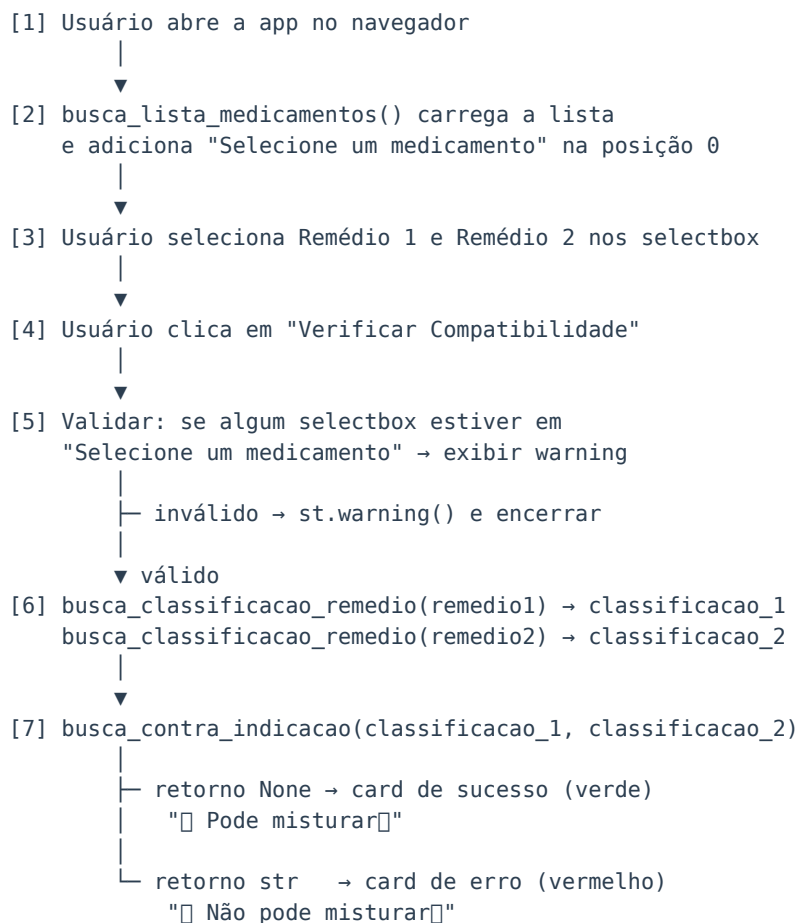
Responsabilidade: consultar se existe uma interação (risco) registrada entre duas classes farmacológicas. A consulta é bidirecional — considera o par (`class_1`, `class_2`) e (`class_2`, `class_1`).

Retorno: `str` com a descrição do risco, ou `None` se não houver interação registrada.

```
def busca_contra_indicacao(classificacao_1, classificacao_2) -> str | None:
    ...
    cursor.execute(f"""
        SELECT risco
        FROM interacao_classes
        WHERE class_1 = '{classificacao_1}' AND class_2 = '{classificacao_2}'
        OR
        class_1 = '{classificacao_2}' AND class_2 = '{classificacao_1}'
    """)
```

6. Fluxo de execução da aplicação

O fluxo de uso do FarmacoMatch segue os passos abaixo:



Pseudocódigo alto nível

```
medicamentos = busca_lista_medicamentos()
remedio1, remedio2 = selecionar_na_ui(medicamentos)

ao clicar_em_verificar:
    se remedio1 ou remedio2 == "Selecione um medicamento":
        exibir_aviso()
    senão:
        c1 = busca_classificacao_remedio(remedio1)
        c2 = busca_classificacao_remedio(remedio2)
        risco = busca_contra_indicacao(c1, c2)
        se risco é None:
            exibir_card_sucesso()
        senão:
            exibir_card_erro()
```

7. Interface e UX

A interface é construída em uma única página com layout `centered` e fonte **Inter** importada via Google Fonts. O estilo customizado é injetado via `st.markdown(..., unsafe_allow_html=True)`.

7.1 Componentes principais

Componente	Tipo Streamlit	Função
Header	HTML/markdown	Título "FarmacoMatch" e subtítulo
Coluna 1	<code>st.selectbox</code>	Seleção do primeiro medicamento
Coluna 2	<code>st.selectbox</code>	Seleção do segundo medicamento
Botão	<code>st.button</code>	Dispara a verificação
Aviso	<code>st.warning</code>	Validar preenchimento
Card sucesso	HTML/markdown	Resultado sem contraindicação
Card erro	HTML/markdown	Resultado com contraindicação

7.2 Layout

```
st.set_page_config(
    page_title="FarmacoMatch",
    page_icon="📄",
    layout="centered",
    initial_sidebar_state="collapsed"
)
```

- Layout centralizado, sidebar recolhida por padrão.
- Dois `selectbox` lado a lado (`st.columns(2, gap="medium")`).
- Botão centralizado via colunas `[1, 2, 1]` (proporção 2/4 do meio).

7.3 Estilo CSS customizado

O arquivo define classes CSS customizadas (não nativas do Streamlit) para refinar a aparência:

- `.main-header / .main-title / .subtitle` — cabeçalho centralizado
- `.stSelectbox` — bordas arredondadas de 12px, sombra suave
- `div.stButton > button` — fundo azul `#3498DB`, hover `#2980B9`, transformação `-2px` no eixo Y
- `.result-card` — padding 1.5rem, raio 15px, animação `fadeInUp`
- `.success-card` — fundo verde `#D4EDDA`, texto `#155724`
- `.error-card` — fundo vermelho `#F8D7DA`, texto `#721C24`
- `@keyframes fadeInUp` — animação de entrada do card (opacidade + translação vertical)

7.4 Paleta de cores

Token Hex	Uso
#2C3E50	Cor do título principal
#7F8C8D	Cor do subtítulo
#34495E	Cor dos rótulos dos inputs
#3498DB	Azul do botão (estado normal)
#2980B9	Azul do botão (hover)
#D4EDDA	Fundo do card de sucesso
#155724	Texto do card de sucesso
#F8D7DA	Fundo do card de erro
#721C24	Texto do card de erro

8. Como executar localmente

Pré-requisitos

- Python 3.10+ (uso de type hint `str` | `None` requer 3.10+)
- pip

Passos

```
# 1. Entrar na pasta do projeto
cd FarmacoMatch

# 2. (Recomendado) Criar e ativar ambiente virtual
python -m venv venv
source venv/bin/activate      # Linux/macOS
# venv\Scripts\activate      # Windows

# 3. Instalar dependências
pip install -r requirements.txt

# 4. Iniciar a aplicação
```

```
streamlit run app.py
```

A aplicação abrirá automaticamente no navegador em <http://localhost:8501>.

Estrutura mínima esperada

O banco `med.db` deve estar no mesmo diretório de `app.py` (paths relativos são utilizados nas funções de acesso a dados).

9. Sugestões de evolução

Apesar do projeto estar funcional, há oportunidades de evolução organizadas por categoria:

9.1 Arquitetura e organização

- **Separar camadas:** extrair acesso a dados (`busca_*`) para um módulo `db.py` e isolando a lógica de negócio de apresentação Streamlit.
- **Centralizar constantes:** nome do banco, mensagens e textos em um módulo `config.py`.
- **Variáveis de ambiente:** usar `.env` para o caminho do banco e parâmetros de configuração.

9.2 Interface / UX

- **Campo de busca textual** nos selectbox para grandes listas (Streamlit `st.selectbox` já possui busca nativa a partir de algumas versões).
- **Exibir a descrição do risco** no card de erro (o campo `risco` retornado pela função não está sendo-renderizado na UI).
- **Mostrar a classe farmacológica** de cada medicamento selecionado como informação de apoio.
- **Modo escuro** acessível via toggle.
- **Responsividade mobile** mais robusta.

9.3 Funcionalidades

- **Verificação de mais de dois medicamentos** simultaneamente.
- **Histórico de consultas** na sessão atual.
- **Exportação** do resultado (PDF/impressão).
- **Integração com base externa** (ex.: DrugBank / ANVISA) para além do SQLite local.

9.4 Dados

- **Atualização do banco** via script de ETL versionado.
- **Versionamento do `med.db`** ou migrações (Alembic/Flyway) para acompanhar evoluções do modelo.
- **Chaves estrangeiras** declaradas entre `medicamentos.classificacao` e `interacao_classes.class_1/class_2`.

- **Coluna** `gravidade` normalizada (ex.: `leve` | `moderada` | `grave`) além do campo textual `risco`.

9.5 Qualidade / DevOps

- **Testes automatizados** (pytest) para as funções de acesso a dados.
- **CI** rodando lint (`ruff` ou `flake8`) e testes a cada commit.
- **Deploy** do app via Streamlit Community Cloud ou contêiner Docker para acesso remoto.
- **Tratamento de exceções** mais granular nas conexões SQLite.

10. Glossário rápido

Termo	Significado
Streamlit	Framework Python para construir apps web data-driven sem frontend tradicional
SQLite	Banco de dados relacional embutido em arquivo único
Selectbox	Componente de menu suspenso do Streamlit (<code>st.selectbox</code>)
Classe farmacológica	Categoria de medicamentos com propriedades terapêuticas similares (ex.: <code>ANALGESICOS</code>)
Interação medicamentosa	Alteração no efeito de um medicamento devido à presença de outro
Contraindicação	Situação em que o uso conjunto é desaconselhado

11. Referências do projeto

Item	Localização
Aplicação	<code>FarmacoMatch/app.py</code>
Banco de dados	<code>FarmacoMatch/med.db</code>
Dependências	<code>FarmacoMatch/requirements.txt</code>
Funções de dados	<code>app.py:12</code> , <code>app.py:20</code> , <code>app.py:40</code>
Estilo CSS	<code>app.py:58-145</code>
Layout da página	<code>app.py:5-10</code>
Renderização do resultado	<code>app.py:184-196</code>